

PROJETO 4: MINIX

Gustavo Borges Peres da Silva¹

¹Departamento de Ciência da Computação – Universidade Federal de Catalão (UFCAT)

gustavo.silva@discente.ufcat.edu.br

Objetivos

- Familiarizar o aluno com o SO MINIX e testes em ambiente de máquina virtual;
- Criar uma chamada de sistema.

1. TAREFA 7 criando uma chamada de sistema (de 22/02/2024 a 25/02/2024)

No MINIX 3 servidores tratam as chamadas de sistema. Conforme visto na unidade 1.1 (Introdução ao MINIX), existem servidores para lidar com diversos aspectos do sistema operacional (gerenciamento de processos, gerenciamento de sistema de arquivos, gerenciamento conexões de rede etc). Para adicionar uma nova chamada de sistema são necessárias duas etapas: escrever um manipulador (handler) de chamada de sistema e escrever uma biblioteca de usuário.

O manipulador de chamada de sistema é uma função que é chamada em resposta a um usuário solicitando uma chamada de sistema. Cada chamada de sistema tem um manipulador. Uma biblioteca do usuário empacota os parâmetros para a chamada de sistema e chama o manipulador no servidor apropriado. Um usuário sempre invoca uma chamada de sistema usando a biblioteca de usuário.

O manipulador de chamadas do sistema deve ser colocado em um servidor apropriado, que por sua vez irá processar uma solicitação do usuário, invocando o manipulador correspondente. É importante escolher o servidor correto para a inclusão da chamada de sistema. Por exemplo, se a chamada de sistema deve atualizar o sistema de arquivo ou estruturas de dados fproc, então o manipulador de chamada de sistema deve ser colocado no servidor FS (no MFS, para sistema de arquivos).

Para esta prática, o professor solicitou para adicionarmos uma chamada de sistema simples no MINIX 3, no servidor PM (gerenciador de processo), a qual irá imprimir a seguinte mensagem: “Eu sou uma chamada de sistema!”.

1.1. Conteúdo de auxílio

Para auxiliar o aluno, esta prática será dividida em algumas etapas – estas etapas devem constar no relatório:

- a) O código fonte para todos os servidores está localizado em `/usr/src/servers`. Cada servidor possui um diretório separado. O gerenciador de processos (PM) está localizado em `/usr/src/servers/pm`. Cada diretório de servidor possui dois arquivos chave: `table.c` e `proto.h`. Estes arquivos contêm a definição para a tabela `call_vec`. A tabela `call_vec` é um vetor de ponteiros de função que é

indexado pelo número da chamada de sistema. Em cada linha, o endereço de um manipulador de chamada de sistema é atribuído para uma entrada na tabela e o índice da entrada é o número da chamada de sistema. **Modifique os arquivos `table.c` e `proto.h` de modo a incluir o manipulador `do_mycall`. Deduza como as entradas para a criação deste manipulador devem ser inseridas nos arquivos mencionados. Use a entrada de número 70 para identificar esta chamada de sistema. Observe que `do_mycall` não recebe parâmetros de entrada. Esta chamada irá apenas imprimir a mensagem: “Eu sou uma chamada de sistema!”. Em `proto.h`, insira o protótipo de sua chamada de sistema na seção destinada ao arquivo `misc.c`.**

- b) A implementação da chamada de sistema deve aparecer em `/usr/src/servers/pm`. Esta pode ser implementada em um arquivo “.c” a parte, porém, para simplificar o processo, iremos inserir sua implementação em `/usr/src/servers/pm/misc.c`. O corpo da chamada deverá conter apenas: `printf(“Eu sou uma chamada de sistema!”);`

`return(OK);`

Para definir o cabeçalho da chamada de sistema, observe o cabeçalho da função `do_proctstat`. Ambos são similares.

- c) Vá ao diretório `/usr/src/servers` e faça um `make`. Isso deverá mostrar erros de compilação caso você tenha realizado alguma alteração incorretamente. Corrija seus erros caso seja necessário.
- d) É hora de mapear a chamada de sistema e seu número (70). Para tanto, você deve ir para o diretório `/usr/src/include/minix` e editar o arquivo `callnr.h` de modo a inserir o mapeamento para a sua chamada de sistema. Chame-a de MYCALL no contexto desse arquivo.
- e) **Para simplificar o procedimento de criação da biblioteca de usuário, crie em `/usr/src/include` o arquivo `mylib.h`. Este arquivo irá executar efetivamente a chamada de sistema criada. O código a ser inserido nessa biblioteca é**

```
#include<lib.h>
```

```
#define mycall _mycall
```

```
#include<unistd.h>
```

```
<<similar ao cabeçalho disponível em (b)>>
```

```
{
```

```
message m;
```

```
return (_syscall(PM_PROC_NR, MYCALL, &m));
```

```
}
```

Não se esqueça de incluir uma entrada para `mylib.h` em `/usr/src/include/Makefile`.

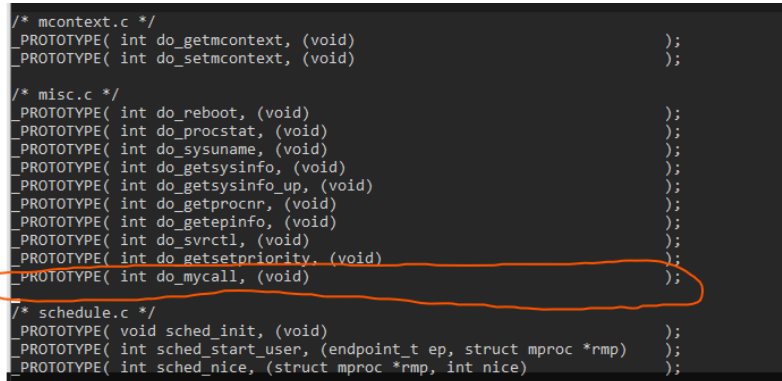
- f) Você observou no código da questão anterior a seguinte declaração: “mensagem”. Explique em que consiste esse tipo de dado. Explique também o papel da chamada de sistema `_syscall()` (obs. Os slides da unidade 2 possuem uma referência a esta questão. Para maiores detalhes consulte o livro Projeto de Sistemas Operacionais que consta na referência do curso).
- g) Para finalizar o processo de criação de sua chamada de sistema, vá ao diretório

/usr/src/tools compile o kernel, bibliotecas e includes e instale no HD.

- h) Crie um programa de teste para sua nova chamada de sistema. Dica: você teria dificuldade em demonstrar a chamada de sistema `printf()`? O raciocínio é análogo.

1.2. Relatório Tarefa 7

Para esta tarefa, iremos seguir os passos descritos no Topico 1.1 Conteúdo de Auxílio, sendo assim, primeiramente iremos editar o arquivo `proto.h` localizado em `/usr/src/servers/pm` no qual iremos adicionar um prototipo de função na seção destinada ao arquivo `misc.c` conforme a figura 1.



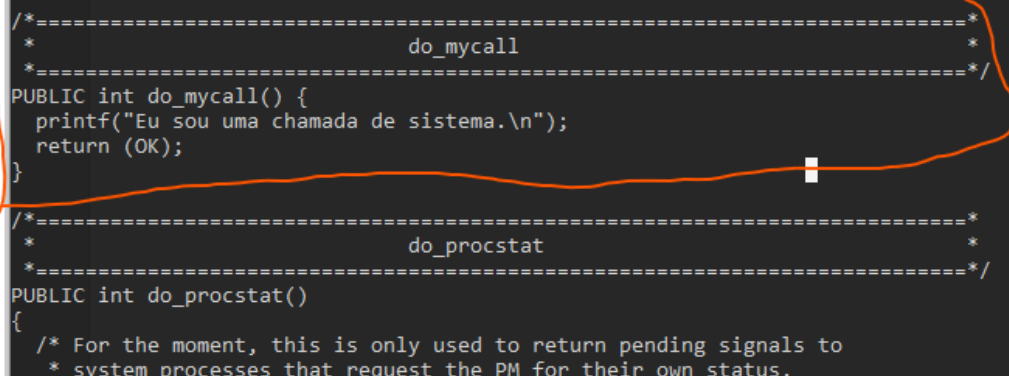
```
/* mcontext.c */
PROTOTYPE( int do_getmcontext, (void) );
PROTOTYPE( int do_setmcontext, (void) );

/* misc.c */
PROTOTYPE( int do_reboot, (void) );
PROTOTYPE( int do_procstat, (void) );
PROTOTYPE( int do_sysuname, (void) );
PROTOTYPE( int do_getsysinfo, (void) );
PROTOTYPE( int do_getsysinfo_up, (void) );
PROTOTYPE( int do_getprocnr, (void) );
PROTOTYPE( int do_getepinfo, (void) );
PROTOTYPE( int do_svrctl, (void) );
PROTOTYPE( int do_getsetpriority, (void) );
PROTOTYPE( int do_mycall, (void) );

/* schedule.c */
PROTOTYPE( void sched_init, (void) );
PROTOTYPE( int sched_start_user, (endpoint_t ep, struct mproc *rmp) );
PROTOTYPE( int sched_nice, (struct mproc *rmp, int nice) );
```

Figura 1: alteração no arquivo de cabeçalho `proto.h`.

Agora que temos o prototipo no cabeçalho, iremos editar o `misc.c` a fins de adicionarmos a implementação da função `do_mycall` conforme a figura 2.



```
/*=====
*
* do_mycall
*=====*/
PUBLIC int do_mycall() {
    printf("Eu sou uma chamada de sistema.\n");
    return (OK);
}

/*=====
*
* do_procstat
*=====*/
PUBLIC int do_procstat()
{
    /* For the moment, this is only used to return pending signals to
    * system processes that request the PM for their own status.
```

Figura 2: alteração no arquivo `misc.c`.

Agora iremos editar o `table.c` a fins de adicionar a entrada da tabela `call_vec` que será a chamada de sistema `do_mycall`. No minix existem algumas tabelas que não estão sendo utilizadas, e uma delas seria essa da posição 70, portanto, iremos

adicionar a entrada na posição 70 da tabela conforme a Figura 3.

```
do_getmcontext, /* 67 = getmcontext */
do_setmcontext, /* 68 = setmcontext */
no_sys,         /* 69 = unused */
do_mycall,      /* 70 = do_mycall */
do_sigaction,   /* 71 = sigaction */
do_sigsuspend,  /* 72 = sigsuspend */
```

Figura 3: alteração no arquivo table.c.

E agora para confirmar que as linhas inseridas em cada documento não deu nenhum problema, iremos usar make no diretório /usr/src/servers conforme a Figura 4 para compilarmos as nossas alterações.

```
root [ /usr/src/servers ]# make
all ==> ds
all ==> ext2
all ==> hgfs
all ==> inet
all ==> init
all ==> ipc
all ==> is
all ==> iso9660fs
all ==> mfs
all ==> pfs
all ==> pm
    create pm/misc.d
    create pm/.depend
    compile pm/misc.o
    compile pm/profile.o
    compile pm/dma.o
    compile pm/mcontext.o
    compile pm/schedule.o
    link pm/pm
all ==> rs
all ==> sched
all ==> vfs
all ==> vm
root [ /usr/src/servers ]#
```

Figura 4: verificando integridade dos arquivos usando make e compilando.

Tudo certo, sendo assim, iremos avançar para o próximo passo, que será o mapeamento da chamada do_mycall com numero 70 no arquivo callnr.h conforme a figura 5, onde o mesmo está localizado no diretório /usr/src/include/minix.

```
#define SETGROUPS      66
#define GETMCONTEXT    67
#define SETMCONTEXT    68

/* Chamada de sistema */
#define MYCALL          70

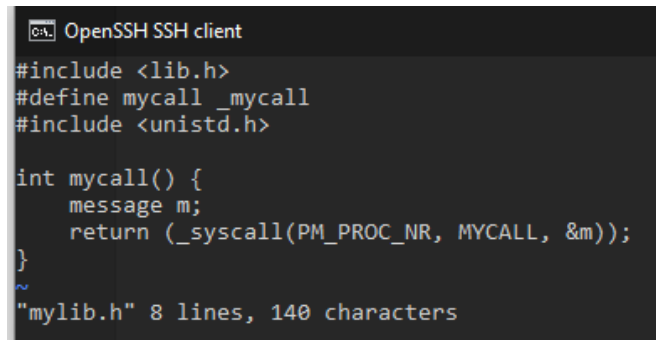
/* Posix signal handling. */
#define SIGACTION       71
#define SIGSUSPEND      72
#define SIGSENDING      73
```

Figura 4: alteração no cabeçalho do arquivo callnr.h.

Agora iremos executar o procedimento para criar uma biblioteca de usuário que irá encapsular a nossa chamada de sistema personalizada. Isso é útil para fornecer uma interface mais amigável e abstrata para os usuários do sistema operacional, em vez de exigir que eles chamem diretamente a chamada de sistema subjacente.

Sendo assim, iremos criar um arquivo chamado **mylib.h** em /usr/src/include e iremos

inserir o conteúdo fornecido pelo professor como podemos observar na Figura 5 abaixo.



```
OpenSSH SSH client

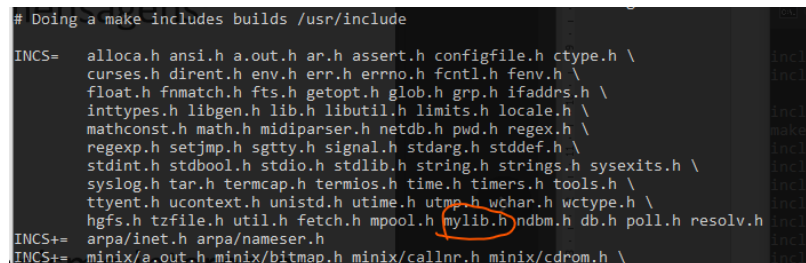
#include <lib.h>
#define mycall _mycall
#include <unistd.h>

int mycall() {
    message m;
    return (_syscall(PM_PROC_NR, MYCALL, &m));
}

~
"mylib.h" 8 lines, 140 characters
```

Figura 5: Conteúdo do arquivo mylib.h.

Além disso, para conseguir compilar, iremos adicionar o mylib.h no arquivo Makefile conforme a Figura 6.



```
# Doing a make includes builds /usr/include

INCS=  alloca.h ansi.h a.out.h ar.h assert.h configfile.h ctype.h \
       curses.h dirent.h env.h err.h errno.h fcntl.h fenv.h \
       float.h fnmatch.h fts.h getopt.h glob.h grp.h ifaddrs.h \
       inttypes.h libgen.h lib.h libutil.h limits.h locale.h \
       mathconst.h math.h midiparser.h netdb.h pwd.h regex.h \
       regexp.h setjmp.h sgtty.h signal.h stdarg.h stddef.h \
       stdint.h stdbool.h stdio.h stdlib.h string.h strings.h sysexits.h \
       syslog.h tar.h termcap.h termios.h time.h timers.h tools.h \
       ttyent.h ucontext.h unistd.h utime.h utmp.h wchar.h wctype.h \
       hgfs.h tzfile.h util.h fetch.h mpool.h mylib.h ndbm.h db.h poll.h resolv.h
INCS+= arpa/inet.h arpa/nameser.h
INCS+= minix/a.out.h minix/bitmap.h minix/callnr.h minix/cdrom.h \
```

Figura 6: Alteração do arquivo Makefile da pasta /usr/src/include.

Agora em resposta da pergunta (f), iremos explicar no que se consiste o uso da declaração “message” no código anterior.

Sendo assim, de acordo com o slide de Introdução ao MINIX, vemos que o MINIX é estruturado em 4 camadas (Figura 7), mas a questão é que, a camada 1 que é realizado o escalonamento dos processos e gerenciamento de transições entre o estados pronto, executando e bloqueado. Além disso, nessa camada o **kernel lida com todas as mensagens entre processos** e controla acesso a portas de I/O e interrupções, tarefas de relógio e tarefas de sistema.

A declaração message no código é um tipo de dado definido no sistema operacional MINIX. Especificamente, é um tipo de dado usado para representar mensagens que são passadas entre processos no sistema operacional.

No contexto do MINIX, a comunicação interprocessos existem três primitivas para enviar e receber mensagens:

- send(dest, &message);
- receive(source, &message);
- sendrec(src_dst, &message);

Ou seja, um processo pode enviar uma mensagem para outro processo usando uma chamada de sistema específica, como **sendrec**. A mensagem pode conter informações como o tipo de mensagem, o processo de origem, o processo de destino e os dados da mensagem propriamente dita.

A chamada de sistema `_syscall` é uma função que permite que um processo faça uma chamada de sistema para o kernel do sistema operacional. Ela é usada para chamar funções do kernel que não estão disponíveis diretamente para o processo de usuário. A chamada de sistema `_syscall` recebe como argumentos o número da chamada de sistema, os parâmetros da chamada de sistema e um ponteiro para uma estrutura de mensagem que contém os dados da chamada de sistema.

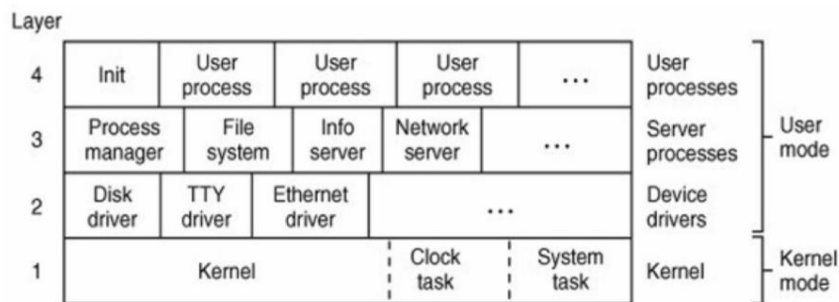


Figura 7: Estrutura interna do MINIX.

Agora que realizamos as explicações, vamos prosseguir. Iremos compilar e instalar o kernel, bibliotecas e includes no MINIX, sendo assim, iremos dividir em algumas etapas.

Na etapa 1, vamos no diretório `/usr/src/tools`, e iremos compilar o kernel, bibliotecas e includes usando os comandos abaixo:

- `make includes` # Instala os arquivos de inclusão
- `make depend` # Gera arquivos de dependência
- `make libraries` # Compila as bibliotecas do sistema
- `make services` # Compila e instala todos os serviços
- `make image` # Cria a imagem de boot

```
root [ /usr/src/tools ]# make includes
make -C .. includes
make -C include includes
install /usr/include/mylib.h
includes ==> sys
includes ==> arch
install /usr/include/machine
includes ==> arch/i386
make -C lib includes
includes ==> csu
includes ==> libc
includes ==> libcurse
includes ==> libdriver
includes ==> libnetdriver
includes ==> libend
includes ==> libedit
includes ==> libm
includes ==> libsys
includes ==> libtimers
includes ==> libutil
```

Figura 8: Trecho onde mostra que `mylib.h` foi instalado.

text	data	bss	size	
98304	39484	349080	486868	kernel
49536	19740	58172	127448	../servers/ds/ds
53200	25304	193180	271684	../servers/rs/rs
48832	20692	381448	450972	../servers/pm/pm
24336	12980	8440	45756	../servers/sched/sched
75232	24912	687832	787976	../servers/vfs/vfs
31584	1735984	16296	1783864	../drivers/memory/memory
32368	15788	96404	144560	../drivers/log/log
60192	25104	146764	232060	../drivers/tty/tty
54240	19300	65920	139460	../servers/mfs/mfs
72928	46388	1451048	1570364	../servers/vm/vm
49744	16736	745292	811772	../servers/pfs/pfs
27984	12616	3488	44088	../servers/init/init
-----	-----	-----	-----	
678480	2015028	4203364	6896872	total

Figura 9: Imagem criada.

Agora na etapa 2, iremos instalar no HD usando o comando `make install`, isso irá criar a imagem de boot e instalará no disco rígido.

```

678480 2015028 4203364 6896872 total
exec sh mkboot hdbboot
install image /dev/c0d0p0s0:/boot/image/3.1.8r10
Done.
root [ /usr/src/tools ]#

```

Figura 10: Alterações instaladas no disco rígido.

Agora iremos reiniciar e executar o nosso boot 2.

Feito a renicialização criamos esse código `teste_mycall.c` para testarmos a chamada de sistema que configuramos. Fizemos um **Makefile** também para compilar o mesmo conforme podemos verificar na Figura 11.

```

#include <stdio.h>
#include <unistd.h>
#include <mylib.h>

int main() {
    int call = mycall();
    if (call == 0) {
        printf("Chamada de sistema executada com sucesso!\n");
    } else {
        printf("Erro ao executar a chamada de sistema.\n");
    }
    return 0;
}

```

Código 1: Conteúdo do arquivo `teste.mycall.c`.

```
OpenSSH SSH client
gustavo [ ~/projeto_4_tarefa7 ]$ ls
Makefile  test_mycall.c
gustavo [ ~/projeto_4_tarefa7 ]$ make
cc -o test_mycall test_mycall.c
"/usr/include/mylib.h", line 5: (warning) '_mycall' old-fashioned function definition
"test_mycall.c", line 5: (warning) 'main' old-fashioned function definition
gustavo [ ~/projeto_4_tarefa7 ]$ ls
Makefile  test_mycall  test_mycall.c
gustavo [ ~/projeto_4_tarefa7 ]$ ./test_mycall
Chamada de sistema executada com sucesso!
gustavo [ ~/projeto_4_tarefa7 ]$ _
```

Figura 11: Compilando o código e testando.

Parece que o nosso programa test_mycall está executando corretamente e a chamada de sistema do_mycall está sendo chamada com sucesso. O aviso sobre a definição de função antiga (old-fashioned function definition) é apenas um aviso e não afeta a execução do programa. Mas após algumas pesquisas, descobrimos que para se livrar desse aviso, podemos atualizar a definição da função do_mycall para seguir o estilo de definição de função mais moderno:

```
/*-----*
 * do_mycall
 *-----*/
PUBLIC int do_mycall(void) {
    printf("Eu sou uma chamada de sistema!\n");
    return (OK); /* OK é uma constante definida em minix/const.h que indica sucesso */
}
```

Figura 12: possível solução para os avisos.

Mas infelizmente, devido o nosso tempo não conseguimos testar essa possível solução para os avisos, mas dando continuidade, iremos mostrar a **ANALOGIA**.

Agora que realizamos os testes finais, podemos dizer que a chamada de sistema mycall foi testada com êxito. O programa de teste simplesmente chama a função mycall e imprime uma mensagem dependendo do resultado da chamada.

No nosso código a gente colocou para imprimir uma mensagem usando printf apenas para mostrar que houve um retorno igual a zero da nossa chamada de sistema, mas como estamos usando o nosso prompt de comando no windows e acessando o MINIX via SSH, precisamos ficar de olho no Sistema do MINIX pois é justamente lá que irá aparecer as mensagens da nossa chamada de sistema.

Assim como a chamada de sistema printf, que é uma chamada de sistema para imprimir texto na tela, a chamada de sistema mycall também é uma chamada de sistema personalizada que criamos para realizar uma tarefa específica. O raciocínio para testar ambas é análogo: chamamos a função correspondente mycall() e verificamos se ela executa a tarefa esperada conforme a Figura 13.

```
OpenSSH SSH client
gustavo [ ~/projeto_4_tarefa7 ]$ ./test_mycall
Chamada de sistema executada com sucesso!
gustavo [ ~/projeto_4_tarefa7 ]$

SO MINIX (Snapshot 1 - com boot) [Executando] - Oracle VM VirtualBox
Arquivo  Máquina  Visualizar  Entrada  Dispositivos  Ajuda
gustavo [ ~ ]$ Eu sou uma chamada de sistema!
OK
```

Figura 13: teste finalizado com sucesso.

Portanto, podemos concluir que a chamada de sistema mycall foi criada e testada com sucesso.

1.3. Salvando os patches da tarefa 7

Para envio da atividade no sigaa, iremos fazer o patch das alterações que realizamos.

Para os arquivos modificados usaremos os seguintes comandos para compararmos os arquivos:

```
diff -c src.clean/servers/pm/proto.h /usr/src/servers/pm/proto.h > Tarefa7.patch
diff -c src.clean/servers/pm/misc.c /usr/src/servers/pm/misc.c >> Tarefa7.patch
diff -c src.clean/servers/pm/table.c /usr/src/servers/pm/table.c >> Tarefa7.patch
diff -c src.clean/include/minix/callnr.h /usr/src/include/minix/callnr.h >> Tarefa7.patch
diff -c src.clean/include/Makefile /usr/src/include/Makefile >> Tarefa7.patch
```

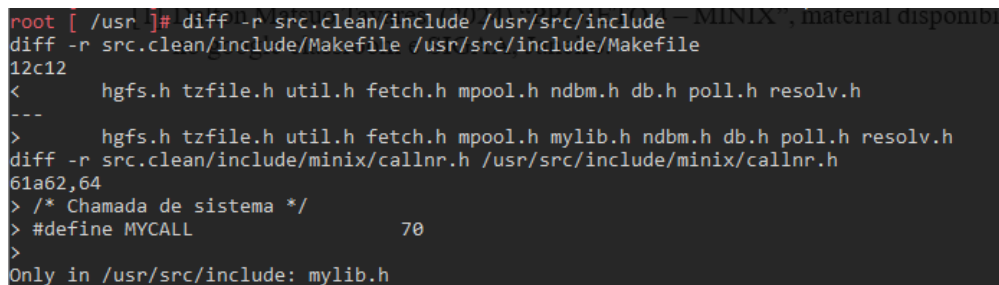
Codigo 2: geração de patches parte 1

E para os arquivos que criamos e não existem no src.clean infelizmente não podemos fazer o patch para eles pois afinal não tem como fazer um patch de um arquivo que não existe no diretório.

Sendo assim, para mostrar as diferenças de arquivos basta usar esse código abaixo, e o resultado será de acordo com a Figura 14.

```
diff -r src.clean/include /usr/src/include
```

Codigo 3: geração de patches parte 2



```
root [ /usr ]# diff -r src.clean/include /usr/src/include - MINIX", material disponibi
diff -r src.clean/include/Makefile /usr/src/include/Makefile
12c12
< hgfs.h tzfile.h util.h fetch.h mpool.h ndbm.h db.h poll.h resolv.h
---
> hgfs.h tzfile.h util.h fetch.h mpool.h mylib.h ndbm.h db.h poll.h resolv.h
diff -r src.clean/include/minix/callnr.h /usr/src/include/minix/callnr.h
61a62,64
> /* Chamada de sistema */
> #define MYCALL 70
>
Only in /usr/src/include: mylib.h
```

Figura 14: Diferença de arquivos.

Como podemos observar, o arquivo mylib.h somente existe no diretório /usr/src/include, então **ANTES de fazer o patch** tem que **copiar-lo para src.test/include**.

Curiosidade, o uso de >> duplicado é para não substituírmos o conteúdo do patch, então isso faz com que usamos apenas um patch para tudo.

Agora que fizemos o patch, iremos fazer uma instalação limpa do nosso kernel através da pasta src.test que copiamos da src.clean, e ao tentarmos executar o nosso código novamente, obtivemos o seguinte status de acordo com a Figura 15.

```
gustavo [ ~ ]$ ls
projeto_4_tarefa7
gustavo [ ~ ]$ cd projeto_4_tarefa7/
gustavo [ ~/projeto_4_tarefa7 ]$ ls
Makefile  test_mycall  test_mycall.c
gustavo [ ~/projeto_4_tarefa7 ]$ ./test_mycall
PM: in no_sys, call nr 70 from 36680
Erro ao executar a chamada de sistema.
```

Figura 15: kernel clean nao roda o nosso codigo - OK.

Isso significa que a instalação está limpa, e de fato o nosso código não vai funcionar no kernel limpo do MINIX.

Agora para confirmar que o patch das alterações vai funcionar, vamos mover o nosso patch para dentro de **/usr/src.teste**, e iremos aplicar as configurações usando **patch -p1 < Tarefa7.patch**.

Caso deia tudo certo não deverá apresentar erros, sendo assim, basta rodar **make clean install** dentro da pasta **/usr/src.teste** e uma vez finalizado basta reiniciar o MINIX.

Após reiniciado iremos testar o nosso código test_mycall.c novamente.

```
gustavo [ ~ ]$ cd
gustavo [ ~ ]$ cd projeto_4_tarefa7/
gustavo [ ~/projeto_4_tarefa7 ]$ ls
Makefile  test_mycall  test_mycall.c
gustavo [ ~/projeto_4_tarefa7 ]$ ./test_mycall
Eu sou uma chamada de sistema.
Chamada de sistema executada com sucesso!
```

Figura 16: patch instalado e funcionando normalmente.

E como podemos observar, o patch funcionou corretamente sem erros.

2. Referências

- [1] Dalton Matsuo Tavares, (2024) “PROJETO 4 – MINIX”, material disponibilizado no google classroom e SIGAA, Janeiro.